

Sistemas Operacionais

Deadlock

Deadlock

- Deadlock
- Condições necessárias para ocorrência de Deadlock
- Grafo de Alocação de Recursos
- Métodos para tratar Deadlock
 - Prevenção
 - Impedimento ou evitar deadlock
 - Detecção e Recuperação

Deadlocks - Impasses

- “Quando dois trens se aproximarem um do outro em um cruzamento, ambos deverão parar completamente e nenhum dos dois deverá dar a partida novamente até que o outro tenha passado.”
- Lei aprovada pela assembleia do estado norte-americano de Kansas, no início do século XX

Deadlocks - Impasses

- Em um ambiente multiprogramado, vários processos podem competir por um número finito de recursos.
- Se um processo requisita recursos e estes não estiverem disponíveis, o processo entra em estado de “espera”. Pode acontecer deste processo nunca mudar de estado, quando os recursos requisitados estão retidos por outros processos no estado de espera.

Deadlocks - Impasses

- A maioria dos sistemas operacionais modernos não provê facilidades de prevenção de deadlock.
- Talvez, sejam incorporados nos sistemas devido a evolução dos SO e das arquiteturas de computadores:
 - Ambientes multiprocessados (multicore), grande quantidade de processos/threads, maior quantidade de recursos no sistema.
- Em um sistema, há uma quantidade finita de recursos a serem usados por vários processos concorrentes.
- Recursos são de diferentes tipos e cada um possui uma quantidade de instâncias:
 - Físico (impressora, fita, memória, CPU)
 - Lógico (variáveis, estruturas, semáforos, arquivos, operações com objetos, monitores)

Deadlock

- Para um processo utilizar um recurso, ele deve solicitá-lo primeiro
- Há assim uma sequência para utilização de um recurso
 - Pedido (se não for satisfeito, o processo que solicitou entra no estado de espera)
 - Uso
 - Liberação
- O pedido e liberação de recursos são chamadas ao sistema (system calls):
 - `request/release device, open/close file, allocate/free memory`

Deadlock

- Um conjunto de processos está em Deadlock quando todos os processos no conjunto estão esperando por um evento ou recurso de outro processo do mesmo conjunto
- Definição:
- n processos estão em deadlock quando cada um dos n estiver bloqueado à espera de um evento que somente pode ser causado por um dos n processos do conjunto.

Deadlock - exemplos

- Mesmo tipo de recursos:
 - Sistema com 3 unidades de fita; há 3 processos, cada um mantendo uma unidade neste momento e para terminar a operação solicitam mais uma unidade de fita.
- Recursos diferentes:
 - P1 alocou a fita; P2 alocou a impressora; para continuar, P1 solicitou a impressora e P2, a fita.
- Em um deadlock os processos nunca terminam sua execução e os recursos do sistema ficam comprometidos impedindo que outros processos continuem.

Condições necessárias para ocorrência de Deadlock

- Para ocorrer um Deadlock, quatro condições devem acontecer simultaneamente:
 - **Exclusão Mútua**
 - Os processos possuem seção crítica de código;
 - A seção crítica deve ser executada com exclusão mútua;
 - Um processo que solicita um recurso compartilhado não poderá utilizá-lo, se este estiver em uso, e deverá esperar até o recurso ter sido liberado.
 - **Posse e Espera**
 - Um processo tem a posse de um recurso e espera por outro recurso alocado a outro processo.

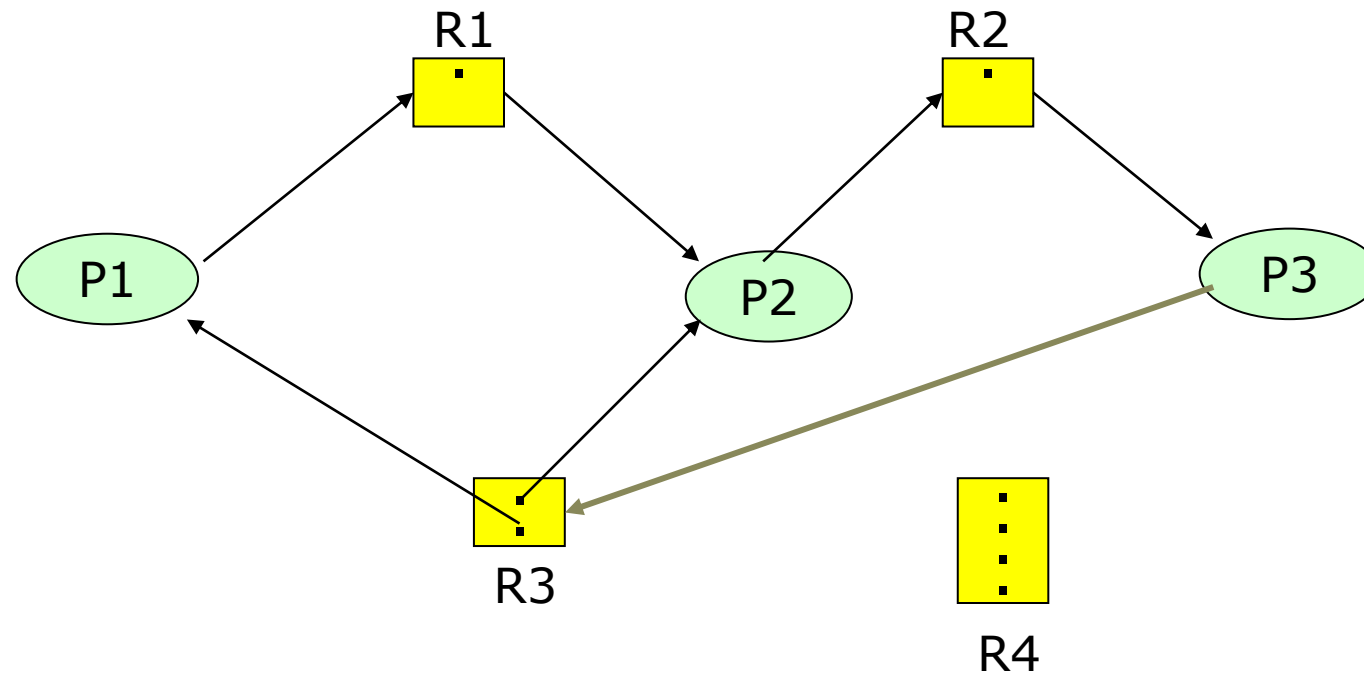
Condições necessárias para ocorrência de Deadlock

- Para ocorrer um Deadlock, quatro condições devem acontecer simultaneamente:
 - Não-preempção
 - Os recursos não podem ser preemptados
 - O processo somente libera o recurso após o término da tarefa.
 - Espera Circular
 - Existe um conjunto de processos e há uma espera circular entre os processos: P0 espera pelo recurso mantido por P1, que espera pelo recurso mantido por P2 , que aguarda pelo recurso Pn-1 que aguarda pelo recurso P0.
 - O ciclo de espera não necessariamente é nesta ordem

Deadlock

- O grafo de alocação de recursos do sistema é utilizado para descrever deadlocks.
- Conjunto de Vértices (processos): $P = \{P1, P2, \dots, Pn\}$
- Conjunto de Recursos: $R = \{R1, R2, \dots, Rn\}$
- Arestas:
 - Pedido: P1 solicitou R1 e está esperando pelo recurso $\rightarrow P1 \rightarrow R1$
Aresta de Pedido
 - Atribuição ou Alocação: R1 foi alocado ao processo P1 $\rightarrow R1 \rightarrow P1$
Aresta de Atribuição

Grafo de Alocação de Recursos - exemplo



Grafo de Alocação de Recursos

- Se um grafo de alocação não possuir um ciclo, o sistema não entrará em deadlock
- Se houver ciclo, o sistema poderá ou não estar em deadlock.
 - Para os sistema estar em deadlock, as 4 condições de ocorrência devem estar presentes.

Métodos para tratar Deadlocks

- Prevenção
 - Garantir que o sistema nunca entrará em deadlock
- Impedimento ou evitar o deadlock
 - Exige que o SO receba informações antecipadas sobre a alocação dos recursos.
- Detecção e Recuperação
 - Permitir que o sistema entre em deadlock:
 - detectá-lo e recuperá-lo
 - ou
 - ignorar o problema e fingir que os deadlocks nunca ocorrem (Unix e Windows)

Prevenção de Deadlock

Prevenção de Deadlock

- A prevenção de deadlock consiste em eliminar uma das 4 condições para ocorrência do deadlock
- Exclusão Mútua
 - Isso é difícil de ser alcançado!
- Posse e Espera
 - Baixa utilização dos recursos
 - Recursos alocados e não utilizados por período longo
 - Possibilidade de paralização

Prevenção de Deadlock

- A prevenção de deadlock consiste em eliminar uma das 4 condições para ocorrência do deadlock
 - Não-preempção
 - Espera Circular
 - Para não ocorrer... impor ordem total sobre todos os tipos de recursos e exigir que cada processo solicite o recurso em ordem ascendente de enumeração
 - Desvantagens:
 - Baixa utilização dos recursos
 - Throughput reduzido (taxa de transmissão reduzida)
 - Paralisação do sistema

Impedimento de Deadlock ou Evitar Deadlock

Impedimento de Deadlock

- Método alternativo para evitar deadlocks
- Processo fornece informações adicionais:
 - como os recursos estão sendo solicitados
- Modelo mais simples e útil de impedimento
 - Cada processo declara o número máximo de recursos de cada tipo
 - Conhecendo isso, pode-se fazer um algoritmo de impedimento de deadlock

Impedimento de Deadlock

- Esse algoritmo examina o estado (disponível, alocado, para quem...) de alocação de recursos para garantir que não aconteça espera circular
- Estado
 - Número de recursos alocados e disponíveis
 - Demanda dos próximos processos

Impedimento de Deadlock

- Estado seguro
- Um sistema está em estado seguro se há uma sequência segura de alocação de recursos (ou seja, não há ciclo no grafo de alocação).
- Um sistema em estado inseguro, não necessariamente está em deadlock,
- porém, um estado com deadlock é um estado inseguro.



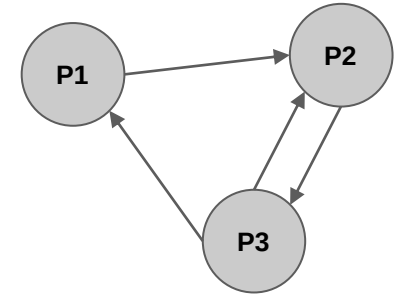
Detecção e Recuperação de Deadlock

Detecção de Deadlock

- Se não é empregada a Prevenção ou o Impedimento de Deadlock no algoritmo, então uma situação de deadlock poderá ocorrer.
- Nesse ambiente, o sistema pode:
 - Detectar Deadlock e
 - Recuperar Deadlock

Detecção de Deadlock

- Utiliza Grafo de Espera
 - mostra a dependência entre os processos
 - as arestas do grafo são somente de processos.
- Existe deadlock se houver um ciclo no grafo de espera
- Algoritmo de procura de ciclos no grafo de espera tem complexidade $O(n^2)$, onde n é a quantidade de vértices no grafo.



Detecção de Deadlock

- Quando o algoritmo de detecção deve ser chamado?
 - Se os deadlocks ocorrem com frequência, esse algoritmo também deve ser chamado com frequência
 - Quando o pedido não pode ser atendido imediatamente (pode ser um deadlock)-> nesse caso, chamar o algoritmo
 - 1 x / hora
 - Utilização da CPU < 40%
- Custo elevado

Recuperação de Deadlock

- Quando detectado o deadlock, pode-se
 - deixar o operador tratar o problema manualmente
 - ou
 - o sistema recupera o deadlock automaticamente:
 - Término do processo (interrompe a espera circular)
 - Preempção de recursos

Término do Processo

- O sistema recupera todos os recursos alocados aos processos terminados.
- O sistema pode:
 - Abortar todos os processos em deadlock
 - Alto custo pois descarta toda a computação até o momento
 - Abortar um processo de cada vez até eliminar o ciclo de deadlock
 - Também possui alto custo porque após cada processo ser abortado, o algoritmo de detecção deve ser chamado
 - Por qual processo começar?

Término do Processo

- Abortar um processo não é tarefa fácil
 - Pode estar no meio de uma operação com arquivos, por exemplo, threads...
- Se o término for parcial é preciso determinar qual(is) processo(s) serão abortados
 - geralmente, opta-se por terminar os processos de custo mínimo

Término do Processo

- Alguns fatores para escolha
 - Qual a prioridade do processo?
 - Quanto tempo de computação já foi realizado e quanto tempo necessita para terminar?
 - Quantos e que tipo de recursos foi utilizado (esses são de fácil preempção)?
 - Quantos recursos o processo ainda precisa?
 - Quantos processos precisarão ser terminados?
 - O processo é interativo ou em batch?

Preempção de Recursos

- Para recuperar deadlock, pode-se fazer a preempção sucessiva de alguns recursos e esses recursos são designados a outros processos até que o ciclo de deadlock seja quebrado
- Três questões devem ser analisadas...

Preempção de Recursos (2)

- Seleção de uma vítima
 - Quais recursos e processos devem ser submetidos a preempção?
 - Determinar a ordem de preempção para minimizar custos (número de recursos mantidos e quantidade de tempo que já consumiu em execução)
- Rollback (reversão)
 - Ao efetuar a preempção em um recurso de um processo, esse deve ser retornado a um estado seguro -> rollback total (abortar o processo e reiniciá-lo)

Preempção de Recursos (3)

- Starvation (paralisação)
 - Como garantir que o processo que sofrerá preempção não será sempre o mesmo, até porque utiliza-se o menor custo? Este processo poderá sofrer paralisação.
 - O processo não consegue terminar sua execução.
 - Incluir o número de rollbacks no custo.

Referências Bibliográficas

- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGME, Greg. Sistemas Operacionais com Java. Rio de Janeiro: Elsevier, 2008.